Estructura de Datos
Unidad 2 – Actividad 1

Sistema de Transformaciones Discretas

Reporte de Resultados

Yahir Alejandro Roque Martinez

Ingenieria en Tecnologias de la Información e Innovación Digital

Universidad Politecnica de Victoria

1. Introducción y Descripción del Problema

Este proyecto implementa un sistema de transformaciones discretas que permite almacenar una función discreta f(n) definida en un intervalo finito [0, N], generar una matriz de transformación T (operador de diferenciación discreta) y aplicar transformaciones sucesivas para obtener las diferencias discretas entre puntos contiguos de la función.

La matriz de transformación T de tamaño N×N se define como:

T(i,j) = 1	si i = j
T(i,j) = -1	si i = j + 1
T(i,j) = 0	en otro caso

El sistema permite aplicar transformaciones sucesivas (primera diferencia, segunda diferencia, etc.), registrar el historial en una lista enlazada genérica, y ofrece funcionalidad de deshacer (undo) mediante una pila y rehacer (redo) mediante una cola. También calcula estadísticas básicas (mínimo, máximo, media) de cada transformación y permite exportar un reporte en texto plano.

Estructuras de datos implementadas

Vector<T>: Implementación propia con memoria dinámica (sin std::vector). Soporta acceso por índice, redimensionamiento, agregar/eliminar elementos y cálculo de estadísticas.

Matriz<T>: Almacenamiento dinámico bidimensional. Soporta generación de la matriz de transformación y multiplicación por vector.

ListaEnlazada<T>: Lista simplemente enlazada genérica. Cada nodo contiene el resultado de una transformación con sus estadísticas y nivel.

Pila<T>: Implementada sobre Vector<T> dinámico. Usada para el sistema de undo.

Cola<T>: Implementada sobre Vector<T> dinámico con compactación automática. Usada para el sistema de redo.

Transformacion: Encapsula el vector resultado, estadísticas (min, max, media) y nivel de transformación.

SistemaTransformaciones: Clase principal que orquesta el menú interactivo y todas las operaciones.

Estructura del proyecto

- El proyecto CMake tiene la siguiente organización:

- CMakeLists.txt – Archivo de configuración CMake
- src/main.cpp – Punto de entrada del programa
- include/Vector.h – Template Vector<T>
- include/Matriz.h – Template Matriz<T>
- include/ListaEnlazada.h – Template ListaEnlazada<T>
- include/Pila.h – Template Pila<T>
- include/Cola.h – Template Cola<T>
- include/Transformacion.h – Clase Transformacion
- include/SistemaTransformaciones.h – Clase principal
- build/ – Directorio de compilación
- docs/ – Documentación y reportes

2. Casos de Prueba Utilizados

Caso 1: Función cuadrática $f(n) = n^2$

Se ingresa la función f(n) = {1, 4, 9, 16, 25} correspondiente a n^2 para n = 1, 2, 3, 4, 5.

Función original:

f = [1, 4, 9, 16, 25]

Matriz de transformación T (5×5):

1	0	0	0	0
-1	1	0	0	0
0	-1	1	0	0
0	0	-1	1	0
0	0	0	-1	1

Primera diferencia (Nivel 1): $g = T \cdot f$

g_1 = [1, 3, 5, 7, 9]

Estadísticas: Min = 1, Max = 9, Media = 5

Segunda diferencia (Nivel 2): $g = T \cdot g_1$

g_2 = [1, 2, 2, 2, 2]

Estadísticas: Min = 1, Max = 2, Media = 1.8

Verificación: La primera diferencia de una función cuadrática produce una función lineal (diferencias impares: 3, 5, 7, 9), y la segunda diferencia produce una constante (2), lo cual confirma el correcto funcionamiento del sistema.

Caso 2: Prueba de undo/redo

Partiendo del caso anterior con el vector actual g_2 = [1, 2, 2, 2, 2]:

Deshacer (undo): Se recupera el estado anterior: [1, 3, 5, 7, 9] (nivel 1). El estado deshecho se envía a la cola de redo.

Rehacer (redo): Se recupera el estado: [1, 2, 2, 2, 2] (nivel 2). El estado se recupera de la cola de redo y el anterior se apila en undo.

Deshacer dos veces: Primero vuelve a nivel 1 [1, 3, 5, 7, 9], luego a nivel 0 (función original) [1, 4, 9, 16, 25].

Caso 3: Función lineal f(n) = 2n + 1

Se ingresa f(n) = {1, 3, 5, 7, 9, 11} para n = 0, 1, 2, 3, 4, 5.

f = [1, 3, 5, 7, 9, 11]

Primera diferencia:

g_1 = [1, 2, 2, 2, 2, 2]

Estadísticas: Min = 1, Max = 2, Media = 1.833

Segunda diferencia:

g_2 = [1, 1, 0, 0, 0, 0]

Estadísticas: Min = 0, Max = 1, Media = 0.333

La primera diferencia de una función lineal produce una constante (2), confirmando el comportamiento esperado del operador de diferenciación discreta.

3. Evidencia de Funcionamiento

A continuación se muestra la salida del programa para el caso de prueba principal (función cuadrática):

```
rocke@debian:~/Documentos/drSaid/ds-u2-a1-Roque31-y/build$ ./transformaciones
 ==========================================
   Estructura de Datos - Unidad 2 - Act 1
   Transformaciones Discretas
 ==========================================

 ============================================
   SISTEMA DE TRANSFORMACIONES DISCRETAS
 ============================================
   1. Ingresar funcion discreta
   2. Generar matriz transformadora
   3. Aplicar transformacion
   4. Mostrar historial de transformaciones
   5. Deshacer ultima transformacion (undo)
   6. Rehacer transformacion (redo)
   7. Calcular estadisticas
   8. Exportar reporte
   9. Salir
 ============================================
   Seleccione una opcion: 1

 Ingrese el numero de puntos de la funcion discreta (N+1): 5
 Ingrese los 5 valores de f(n):
   f(0) = 1
   f(1) = 2
   f(2) = 9
   f(3) = 16
   f(4) = 25

 Funcion ingresada correctamente: [ 1, 2, 9, 16, 25 ]
```

```
 ============================================
   Seleccione una opcion: 2

 Matriz de transformacion (5x5) generada:
 |      1      0      0      0      0 |
 |     -1      1      0      0      0 |
 |      0     -1      1      0      0 |
 |      0      0     -1      1      0 |
 |      0      0      0     -1      1 |
```

```
   8. Exportar reporte
   9. Salir
 ============================================
   Seleccione una opcion: 3

 Transformacion nivel 1 aplicada.
 Resultado: [ 1, 1, 7, 7, 9 ]
 Min: 1 | Max: 9 | Media: 5

 ============================================
```

```
 ============================================
   Seleccione una opcion: 4

 ========================================
    HISTORIAL DE TRANSFORMACIONES
 ========================================

 Funcion original: [ 1, 2, 9, 16, 25 ]

 Nivel 1: [ 1, 1, 7, 7, 9 ]
   Min: 1.0000 | Max: 9.0000 | Media: 5.0000

 Vector actual: [ 1.0000, 1.0000, 7.0000, 7.0000, 9.0000 ]
 ========================================
```

```
  ==========================================
    Seleccione una opcion: 7

  ========================================
         ESTADISTICAS
  ========================================

  Funcion original:
    Valores: [ 1.0000, 2.0000, 9.0000, 16.0000, 25.0000 ]
    Minimo:  1.0000
    Maximo:  25.0000
    Media:   10.6000

  Transformaciones:
    Nivel 1:
      Minimo:  1.0000
      Maximo:  9.0000
      Media:   5.0000

  Vector actual:
    Valores: [ 1.0000, 1.0000, 7.0000, 7.0000, 9.0000 ]
    Minimo:  1.0000
    Maximo:  9.0000
    Media:   5.0000
  ========================================
```

reporte_transformaciones.txt

```
==============================================================
  REPORTE DE TRANSFORMACIONES DISCRETAS
==============================================================

FUNCION ORIGINAL:
  Valores: [ 1.0000, 2.0000, 9.0000, 16.0000, 25.0000 ]
  Tamano:  5
  Minimo:  1.0000
  Maximo:  25.0000
  Media:   10.6000

MATRIZ DE TRANSFORMACION:
| 1.0000 0.0000 0.0000 0.0000 0.0000 |
| -1.0000 1.0000 0.0000 0.0000 0.0000 |
| 0.0000 -1.0000 1.0000 0.0000 0.0000 |
| 0.0000 0.0000 -1.0000 1.0000 0.0000 |
| 0.0000 0.0000 0.0000 -1.0000 1.0000 |

HISTORIAL DE TRANSFORMACIONES:
--------------------------------------------------------------
  Nivel 1:
    Resultado: [ 1.0000, 1.0000, 7.0000, 7.0000, 9.0000 ]
    Minimo:    1.0000
    Maximo:    9.0000
    Media:     5.0000

--------------------------------------------------------------
VECTOR ACTUAL:
  Valores: [ 1.0000, 1.0000, 7.0000, 7.0000, 9.0000 ]
  Minimo:  1.0000
  Maximo:  9.0000
  Media:   5.0000

Nivel de transformacion actual: 1
Transformaciones en pila undo: 1
Transformaciones en cola redo: 0
==============================================================
```

4. Conclusiones sobre el Uso de Estructuras de Datos

Vector<T> (Arreglo dinámico): La implementación propia del vector con memoria dinámica permitió comprender a fondo la gestión manual de la memoria (new/delete), el redimensionamiento y la importancia del constructor copia y operador de asignación (regla de los tres). Esta estructura fue la base sobre la cual se construyeron la Pila y la Cola, demostrando la versatilidad de los arreglos dinámicos como estructura fundamental.

Matriz<T> (Arreglo bidimensional): La matriz dinámica bidimensional demostró ser adecuada para representar el operador de diferenciación discreta. La operación de multiplicación matriz-vector fue esencial para aplicar las transformaciones. Se observó que la matriz de transformación es dispersa (la mayoría de elementos son cero), lo cual sugiere que en aplicaciones reales se podrían usar representaciones dispersas para mayor eficiencia.

ListaEnlazada<T>: La lista enlazada resultó ideal para almacenar el historial de transformaciones, ya que permite agregar nuevos nodos sin necesidad de redimensionar. Cada nodo encapsula un resultado completo (vector + estadísticas + nivel), validando la utilidad de las listas para almacenar secuencias de longitud variable. La implementación genérica con templates permitió reutilizar la estructura con cualquier tipo de dato.

Pila<T> (Undo): La pila, implementada sobre el vector dinámico, fue la estructura natural para el sistema de deshacer (undo), ya que la operación de deshacer sigue el principio LIFO (Last In, First Out): siempre se deshace la última transformación aplicada. Esto demuestra cómo las pilas modelan perfectamente operaciones reversibles en orden inverso.

Cola<T> (Redo): La cola, también implementada sobre el vector dinámico, se utilizó para el sistema de rehacer (redo), siguiendo el principio FIFO (First In, First Out): las transformaciones deshechas se rehacen en el mismo orden en que fueron originalmente aplicadas. La compactación automática de la cola previene el desperdicio de memoria cuando se desencolan muchos elementos.

Programación genérica (Templates): El uso de plantillas (template<class T>) permitió crear estructuras de datos reutilizables e independientes del tipo. Esto facilitó que las mismas clases Vector, Lista, Pila y Cola pudieran trabajar tanto con tipos numéricos como con objetos Transformacion, demostrando el poder de la programación genérica en C++.

Conclusión general: El proyecto demostró que la correcta selección de estructuras de datos es fundamental para resolver problemas computacionales de manera eficiente y elegante. Cada estructura (vector, matriz, lista enlazada, pila y cola) cumplió un rol específico en el sistema, y su implementación manual desde cero reforzó la comprensión profunda de cómo funcionan internamente estas abstracciones fundamentales de la programación.